Programming Assignment 2

Krish Agarwal

March 24, 2026

1. Methodology

The aim of this exercise is to implement code (which I have done in Python) to display the sampling distributions of three different datasets, and use an estimator (both point and interval) to calculate a guess for the unknown parameter. For the purposes of this assignment, I have considered the unknown parameter to be the mean of the population. As we have shown in class:

$$\mu_{\text{MLE}} = \bar{X} = \frac{\sum_{i=0}^{n} x_i}{n} \tag{1}$$

Where, in (1), n denotes the size of the sample $\underline{x}$. The datasets I have considered were relatively large (containing more than 10,000 entries); therefore I have adopted a considerably large sample for each calculation. The interval estimate is similarly calculated using the formula we have determined in class (since we don't know the true variance, we have substituted the sample variance considering the t-distribution derivation route):

$$\mu_{\text{interval}} \in \left[\bar{X} - \frac{S}{\sqrt{n}} t_{\alpha/2,n-1}, \bar{X} + \frac{S}{\sqrt{n}} t_{\alpha/2,n-1}\right] \tag{2}$$

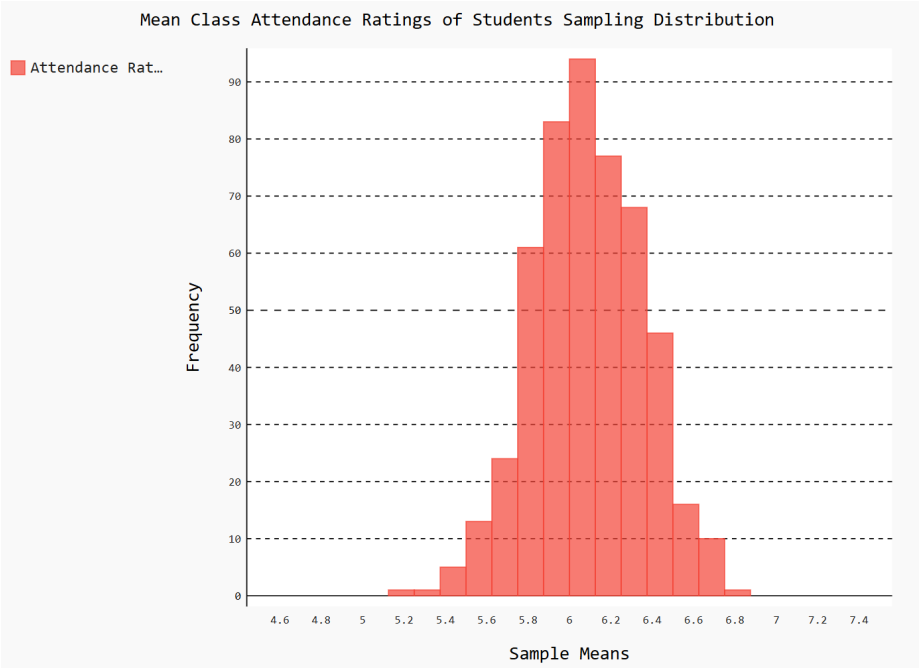
The value of $t_{\alpha/2,n-1}$ is passed as a parameter, considering the size of samples as 50, and for $\alpha = 0.25, 0.5$ across examples.

Finally, the plot for the sampling distribution is given by considering 500 random samples from each dataset and constructing a histogram of these values to get a relative idea of the actual distribution of $\bar{X}$ obtained across the experiments. The images for the same are compiled and saved using the Pygal library. As one would expect from being a direct consequence of the Central Limit Theorem, the distributions take on a Gaussian shape, evident from how they appear.

2. Student Attendance

A dataset containing the attendance percentage of each student in an institute.

2.1. Sampling Distribution:



2.2. Point and Interval Estimate:

```
krishagarwal8959@KrishLaptop:~/PyProj/ProgrammingAssignment2$ python3 MA24BTECH11014_ClassAttendance.py
Real Mean: 5.9854115. Sample Mean: 6.42
90% Interval Estimate: [5.3535577871720115, 7.486442212827988]
```

2.3. Analysis

As expected, the sampling distribution of $\bar{X}$ is a normal curve, that seems to be centered around 6.1. Although this is just a coarse version of the true distribution, it gives us insight into how the CLT works. Also, the sample mean is near enough to the real mean (although some degree of deviation is expected when just taking one sample). The interval estimator indeed contains the real mean here.

2.4. Code (Python):

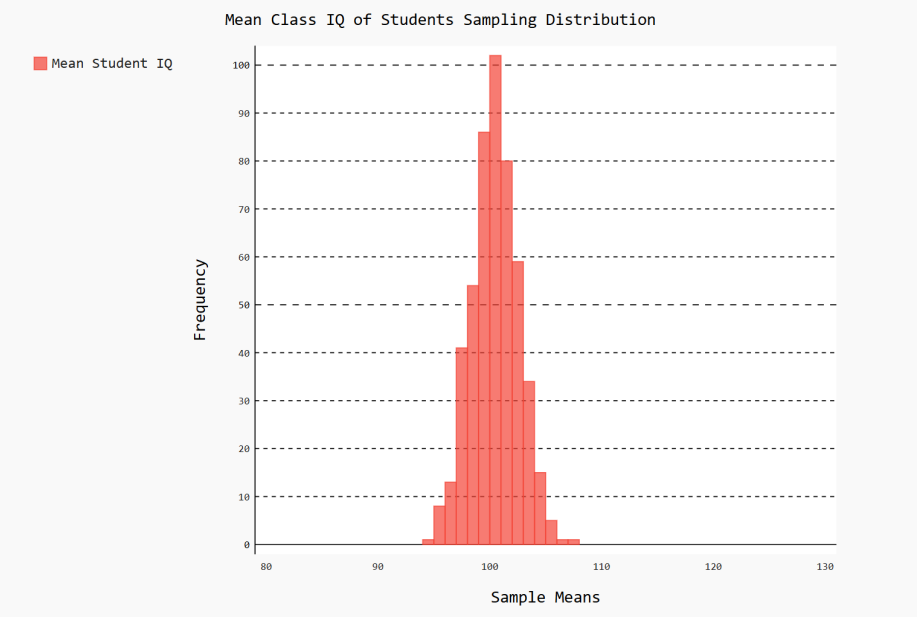
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pygal.style import Style
import pygal
import csv

def get_mean(sample):
    sum = 0
    for x in sample:
        sum += x
    sum /= len(sample)
    return sum

def get_variance(sample):
    square_sum = 0
    mean = get_mean(sample)
    for x in sample:
        square_sum += (x - mean) * (x - mean)
    variance = square_sum/(len(sample) - 1)
    return variance

filename = 'student_performance.csv'

with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)
    participation = []
    for row in reader:
        high = float(row[6])
        participation.append(high)

# sampling distribution of attendance
# lets get a sample of 50 data points, 500 times, and plot them
sample_means = []
for i in range(500):
    random_sample = np.random.choice(participation, 50)
    ans = get_mean(random_sample)
    ans = np.ceil(ans * 8) / 8
    sample_means.append(ans)

mean_labels = []
mean_frequencies = []
```

```
for value in np.arange(4.5, 7.5, 0.125):
    mean_labels.append(value)
    frequency = sample_means.count(value)
    mean_frequencies.append(frequency)

mean_freq_table = pygal.Histogram(title="Mean Class Attendance Ratings of
        Students Sampling Distribution", x_title='Sample Means',
            y_title='Frequency',xrange=(0, 10), yrange=(0, 200))
mean_freq_table.add("Attendance Rating", [(mean_frequencies[ct],
        mean_labels[ct], mean_labels[ct] + 0.125)
            for ct in range(0, 24)])
mean_freq_table.render_to_file('histogram.svg')


# point estimate of mean
real_mean = get_mean(participation)
random_sample = np.random.choice(participation, 50)
sample_mean = get_mean(random_sample)
sample_variance = get_variance(random_sample)

print(f"Real Mean: {real_mean}. Sample Mean: {sample_mean}")

#interval estimate of mean
# we can see that the sample means are roughly normally distributed
t_value = 1.67655
interval_low = sample_mean - t_value * sample_variance / (7)
interval_high = sample_mean + t_value * sample_variance / (7)

print(f"90% Interval Estimate: [{interval_low}, {interval_high}]")
```

3. Student IQ

A dataset containing the measured IQ of each student across many institutes.

3.1. Sampling Distribution:



3.2. Point and Interval Estimate:

```
krishagarwal8959@KrishLaptop:~/PyProj/ProgrammingAssignment2$ python3 MA24BTECH11014_ClassIQ.py
Real Mean: 99.4718. Sample Mean: 97.54
Sample Variance: 188.4575510204082
95% Interval Estimate: [43.43706780291544, 151.64293219708458]
```

3.3. Analysis

Once again, the sampling distribution of $\bar{X}$ is a normal curve. It is noticably dense around 100, which is what is the typically assumed baseline IQ. Also, the sample mean is very near to the real mean (although some degree of deviation is expected when just taking one sample), showing the power of the MLE estimator. The interval estimator indeed contains the real mean here, though aiming for 95% confidence gives us a very very broad interval in this case.

3.4. Code (Python):

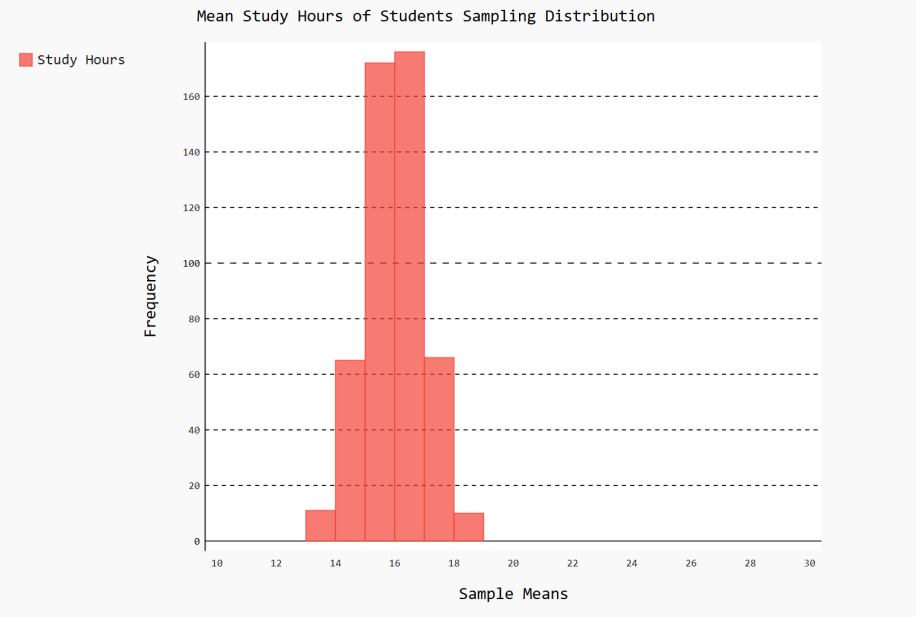
```
     import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pygal.style import Style
import pygal
import csv

def get_mean(sample):
    sum = 0
    for x in sample:
        sum += x
    sum /= len(sample)
    return sum

def get_variance(sample):
    square_sum = 0
    mean = get_mean(sample)
    for x in sample:
        square_sum += (x - mean) * (x - mean)
    variance = square_sum/(len(sample) - 1)
    return variance

filename = 'college_student_placement_dataset.csv'

with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)
    iq = []
    for row in reader:
        curr_iq = int(row[1])
        iq.append(curr_iq)

# sampling distribution of attendance
# lets get a sample of 50 data points, 500 times, and plot them
sample_means = []
for i in range(500):
    random_sample = np.random.choice(iq, 50)
    ans = get_mean(random_sample)
    ans = np.ceil(ans)
    sample_means.append(ans)

mean_labels = []
mean_frequencies = []
```

```
for value in range(80, 130, 1):
    mean_labels.append(value)
    frequency = sample_means.count(value)
    mean_frequencies.append(frequency)

mean_freq_table = pygal.Histogram(title="Mean Class IQ of Students
    Sampling Distribution", x_title='Sample Means', y_title='Frequency',
        xrange=(80, 130), yrange=(0, 100))
mean_freq_table.add("Mean Student IQ", [(mean_frequencies[ct], mean_labels[ct],
    mean_labels[ct] + 1) for ct in range(0, 50)])
mean_freq_table.render_to_file('histogramIQ.svg')


# point estimate of mean
real_mean = get_mean(iq)
random_sample = np.random.choice(iq, 50)
sample_mean = get_mean(random_sample)
sample_variance = get_variance(random_sample)

print(f"Real Mean: {real_mean}. Sample Mean: {sample_mean}")
print(f"Sample Variance: {sample_variance}")

#interval estimate of mean
# we can see that the sample means are roughly normally distributed
t_value = 2.00958
interval_low = sample_mean - t_value * sample_variance / (7)
interval_high = sample_mean + t_value * sample_variance / (7)

print(f"95% Interval Estimate: [{interval_low}, {interval_high}]")
```

4. Student Study Hours

A dataset containing the average study hours of a student in one week.

4.1. Sampling Distribution:



4.2. Point and Interval Estimate:

```
krishagarwal8959@KrishLaptop:~/PyProj/ProgrammingAssignment2$ python3 MA24BTECH11014_ClassStudy.py
Real Mean: 15.029126900000232. Sample Mean: 14.561999999999996
90% Interval Estimate: [0.3649071469679299, 28.75909285303206]
```

4.3. Analysis

The sampling distribution of $\bar{X}$ is a normal curve, though quite sharp near 16. Also, the sample mean is very near to the real mean again, showing that the MLE estimator is indeed a good choice. The interval estimator indeed contains the real mean here, though even just aiming for 90% confidence gives us a broad interval in this case.

4.4. Code (Python):

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pygal.style import Style
import pygal
import csv

def get_mean(sample):
    sum = 0
    for x in sample:
        sum += x
    sum /= len(sample)
    return sum

def get_variance(sample):
    square_sum = 0
    mean = get_mean(sample)
    for x in sample:
        square_sum += (x - mean) * (x - mean)
    variance = square_sum/(len(sample) - 1)
    return variance

filename = 'student_performance.csv'

with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)
    participation = []
    for row in reader:
        high = float(row[1])
        participation.append(high)

# sampling distribution of attendance
# lets get a sample of 50 data points, 500 times, and plot them
sample_means = []
for i in range(500):
    random_sample = np.random.choice(participation, 50)
    ans = get_mean(random_sample)
    ans = np.ceil(ans)
    sample_means.append(ans)

mean_labels = []
mean_frequencies = []
```

```
for value in range(10, 30):
    mean_labels.append(value)
    frequency = sample_means.count(value)
    mean_frequencies.append(frequency)

mean_freq_table = pygal.Histogram(title="Mean Study Hours of Students Sampling
    'Distribution", x_title='Sample Means', y_title='Frequency',
        xrange=(10, 30), yrange=(0, 200))
mean_freq_table.add("Study Hours", [(mean_frequencies[ct - 10],
    mean_labels[ct - 10], mean_labels[ct - 10] + 1) for
        ct in range(10, 30)])
mean_freq_table.render_to_file('histogramStudy.svg')


# point estimate of mean
real_mean = get_mean(participation)
random_sample = np.random.choice(participation, 50)
sample_mean = get_mean(random_sample)
sample_variance = get_variance(random_sample)

print(f"Real Mean: {real_mean}. Sample Mean: {sample_mean}")

#interval estimate of mean
# we can see that the sample means are roughly normally distributed
t_value = 1.67655
interval_low = sample_mean - t_value * sample_variance / (7)
interval_high = sample_mean + t_value * sample_variance / (7)

print(f"90% Interval Estimate: [{interval_low}, {interval_high}]")
```